

Documento Técnico

*Simulador de Gestión de Memoria Paginada usando Árboles AVL y Splay
Trees*

Curso: Estructuras de Datos

Proyecto: Proyecto II

Estudiante: Angie Mariela Alpizar Porras

Semestre: I Semestre 2026

Fecha de entrega: 27 de mayo de 2026

1. Introducción

Este documento describe el diseño y la implementación del **Simulador de Gestión de Memoria Paginada**, un programa escrito en **C++** que reproduce de manera simplificada el comportamiento de un sistema operativo al administrar la memoria principal mediante el esquema de **paginación**. Los procesos del sistema solicitan, acceden y liberan memoria, y el simulador se encarga de mantener actualizadas las páginas físicas libres, las tablas de páginas por proceso y dos niveles de caché (local y global) con política de reemplazo **LRU** (*Least Recently Used*).

El proyecto utiliza de forma central dos estructuras de datos: el **Árbol AVL** y el **Splay Tree**. La elección de estas estructuras no es accidental: cada una cubre una necesidad distinta del simulador. El AVL ofrece búsquedas, inserciones y eliminaciones balanceadas en $O(\log n)$, lo cual es ideal para administrar el conjunto de páginas físicas libres. El Splay Tree, por su parte, lleva a la raíz cualquier nodo recientemente accedido, lo que se acopla naturalmente al concepto de *localidad temporal* presente en los patrones de acceso a memoria.

El programa funciona como una **interfaz de comandos** (consola o archivo de texto) que recibe operaciones del usuario y reporta el resultado, el estado interno y un conjunto de métricas obligatorias. Esta forma de operación es deliberadamente simple, ya que el objetivo del proyecto no es construir una interfaz gráfica sino demostrar el funcionamiento de las estructuras de datos y la correcta integración entre ellas.

2. Descripción del problema

Un sistema operativo administra una cantidad finita de memoria principal dividida en **páginas físicas** de tamaño fijo. Cada proceso que se ejecuta en el sistema solicita memoria, y el administrador debe asignarle páginas físicas. Como los procesos no acceden a posiciones absolutas sino a un espacio de direcciones virtual propio, cada uno requiere una **tabla de páginas** que traduzca su dirección virtual a una dirección física.

Para acelerar los accesos repetidos, los sistemas reales utilizan **memorias caché** que almacenan las traducciones más recientes. En este proyecto se simulan dos niveles:

- Un **caché local** propio de cada proceso, con tamaño máximo proporcional a las páginas que se le han asignado.
- Un **caché global** compartido entre todos los procesos, con tamaño parametrizable.

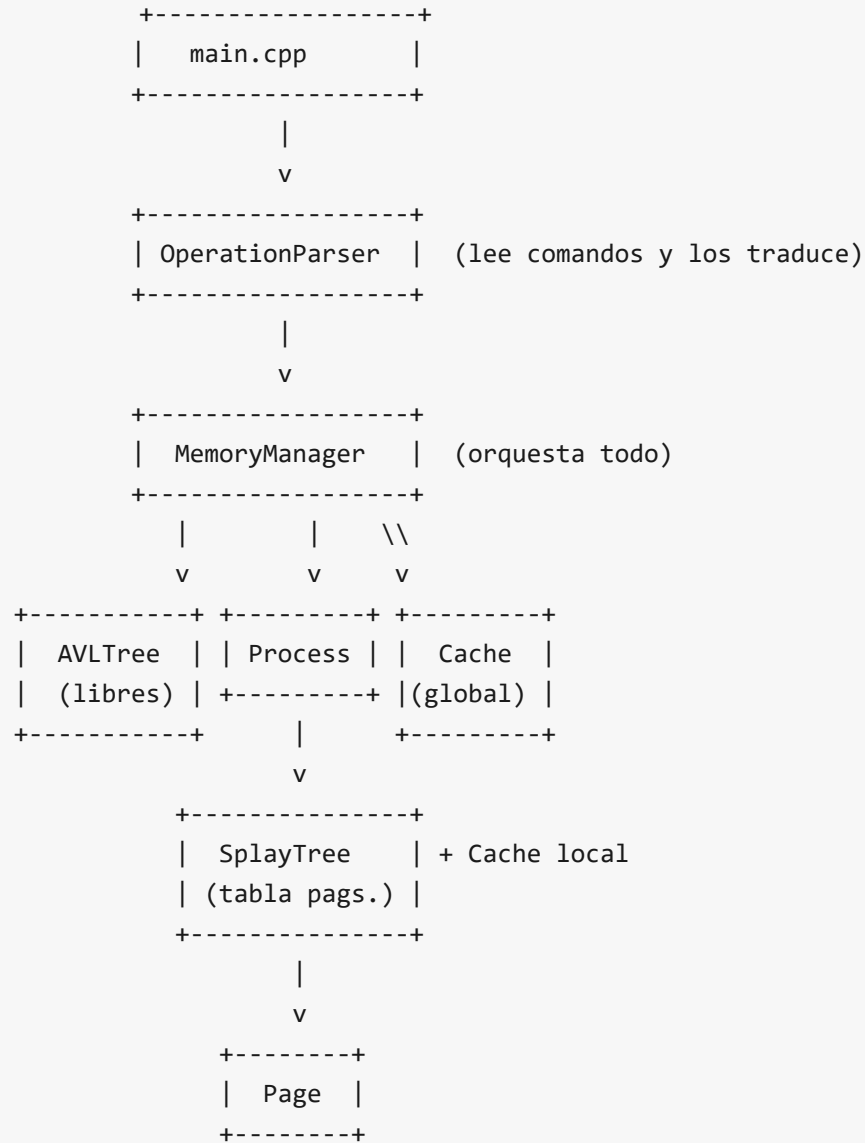
Cuando un caché se llena, se aplica una política de reemplazo que selecciona la víctima a desalojar. En este simulador se implementa la política **LRU**.

El programa debe responder a cinco operaciones principales: crear un proceso (`NEW_PROCESS`), finalizarlo (`END_PROCESS`), acceder a una dirección de memoria (`ACCESS`), solicitar memoria adicional (`ALLOCATE`) y liberar memoria (`FREE`). Adicionalmente debe llevar un conjunto de **métricas** (accesos totales, hits y misses por caché, reemplazos LRU, páginas libres y ocupadas, utilización) y detectar **errores** comunes (procesos inexistentes, direcciones inválidas, memoria insuficiente, entre otros).

3. Diseño de clases

El simulador se compone de siete clases principales, distribuidas en archivos de cabecera (`include/`) y archivos fuente (`src/`). La separación favorece la modularidad, el reemplazo independiente de cada componente y la testeabilidad.

3.1 Diagrama lógico de dependencias



3.2 Descripción por clase

Clase	Archivo	Responsabilidad
Page	include/Page.h	Estructura de datos POD que representa una página de memoria (virtual y/o física). Mantiene bit de validez, bit de modificación, contador de referencias y <i>timestamp</i> del último acceso.
AVLTree<K,V>	include/AVLTree.h	Plantilla del árbol AVL genérico. Provee insertar, eliminar, buscar, inOrden, altura, rotarIzquierda, rotarDerecha y balancear.

Clase	Archivo	Responsabilidad
<code>SplayTree<K,V></code>	<code>include/SplayTree.h</code>	Plantilla del Splay Tree genérico. Provee <code>insertar</code> , <code>eliminar</code> , <code>buscar</code> , <code>recorrido</code> , <code>splay</code> , <code>rotacionZig</code> , <code>rotacionZigZig</code> y <code>rotacionZigZag</code> .
<code>Cache</code>	<code>include/Cache.h</code>	Caché con política LRU implementado con lista doblemente enlazada (orden por uso) e <code>unordered_map</code> (índice). Soporta inserción, búsqueda, eliminación, eliminación masiva por proceso y desalojo automático.
<code>Process</code>	<code>include/Proceso.h</code>	Modela un proceso. Contiene su tabla de páginas (Splay), su caché local, contadores y métodos para añadir y remover páginas.
<code>MemoryManager</code>	<code>include/MemoryManager.h</code>	Núcleo del simulador. Mantiene el AVL de páginas libres, la tabla global de procesos (<code>unordered_map<string, Process></code>), el caché global y las métricas. Implementa las cinco operaciones obligatorias.
<code>OperationParser</code>	<code>include/OperationParser.h</code>	Lee comandos línea por línea desde un <code>std::istream</code> (consola o archivo), los descompone en tokens y los traduce a llamadas sobre <code>MemoryManager</code> .

3.3 Justificación del nombre `Proceso.h`

El archivo de la clase `Process` se llamó **`Proceso.h`** (no `Process.h`) para evitar un conflicto con el header `<process.h>` del CRT de Microsoft Windows. Dado que el sistema de archivos de Windows es *case-insensitive*, al incluir indirectamente `<process.h>` a través de `<iostream>` el preprocesador resolvía la ruta hacia el archivo del proyecto en lugar de hacia el header del sistema, generando errores de compilación. La clase interna conserva el nombre **`Process`** por compatibilidad con el enunciado del proyecto.

4. Estructuras de datos utilizadas

Estructura	Implementación	Uso en el simulador
Árbol AVL	<code>AVLTree<int, Page></code> propia	Conjunto de páginas físicas libres del sistema.

Estructura	Implementación	Uso en el simulador
Splay Tree	<code>SplayTree<int, Page></code> propia	Tabla de páginas de cada proceso (clave = página virtual).
Lista doblemente enlazada	<code>std::list<CacheEntry></code>	Orden de uso de las entradas del caché (LRU).
Tabla hash	<code>std::unordered_map<string, list::iterator></code>	Índice del caché para acceso $O(1)$ a la entrada por clave.
Tabla hash de procesos	<code>std::unordered_map<string, Process></code>	Procesos vivos del sistema (clave = id del proceso).

Conforme a la regla 1.1.17 del enunciado, ni `std::map` ni `std::set` se utilizan como sustituto del AVL o del Splay Tree.

5. Justificación del Árbol AVL

El AVL se eligió para administrar el **conjunto de páginas físicas libres** por tres razones específicas:

1. **Garantía logarítmica en el peor caso.** Un BST sin balanceo puede degradar a $O(n)$ en patrones de inserción ordenada (por ejemplo, al liberar páginas en orden creciente). El AVL mantiene su altura en $O(\log n)$ mediante rotaciones, lo cual asegura inserción, búsqueda y eliminación en tiempo logarítmico aunque las páginas se liberen y reasignen en cualquier orden.
2. **Recuperación eficiente del mínimo.** El simulador asigna siempre la página libre con número más bajo, lo que mantiene la memoria *compacta* (las páginas ocupadas se agrupan al inicio). Esto se obtiene en $O(\log n)$ bajando por los hijos izquierdos del AVL.
3. **Operaciones en orden.** La función `inOrden` permite recorrer las páginas libres en orden ascendente, útil para reportes y para verificar la corrección del estado interno.

El AVL implementado provee los métodos exigidos por el enunciado: `insertar`, `eliminar`, `buscar`, `rotarIzquierda`, `rotarDerecha`, `balancear`, `inOrden`, `altura`. Las rotaciones contemplan los cuatro casos clásicos (izquierda-izquierda, izquierda-derecha, derecha-derecha y derecha-izquierda) y la altura se mantiene incrementalmente en cada nodo.

6. Justificación del Splay Tree

El Splay Tree se eligió para implementar la **tabla de páginas de cada proceso**. La motivación principal es la **localidad temporal**: en la práctica los programas tienden a acceder repetidamente a las mismas páginas en intervalos cortos, comportamiento característico de bucles, recorridos y reuso de variables.

El Splay Tree, al realizar la operación `splay` sobre cualquier nodo accedido, lo lleva a la raíz mediante rotaciones Zig, Zig-Zig y Zig-Zag. Esto produce dos efectos beneficiosos:

- 1. **$O(\log n)$ amortizado** para todas las operaciones sin necesidad de almacenar información de balanceo (no requiere alturas ni factores como el AVL).
- 2. **$O(1)$ amortizado** para accesos a páginas recientemente consultadas, lo que coincide con el patrón de localidad temporal mencionado.

El Splay Tree implementado provee los métodos exigidos: `insertar`, `eliminar`, `buscar`, `splay`, `rotacionZig`, `rotacionZigZig`, `rotacionZigZag` y `recorrido`. La eliminación utiliza el método clásico de hacer splay sobre la clave, separar los subárboles izquierdo y derecho, y unirlos haciendo splay del máximo del subárbol izquierdo.

7. Formato de entrada

El simulador admite operaciones provenientes de dos fuentes equivalentes: **consola** (modo interactivo) y **archivo de texto** (modo batch). En ambos casos cada línea representa una operación y el conjunto soportado es el siguiente:

Comando	Sintaxis	Descripción
<code>MEMORIA_TOTAL n</code>	<code>n ≥ 1</code>	Memoria total del sistema en bytes
<code>TAMANO_PAGINA n</code>	<code>n ≥ 1</code>	Tamaño de página en bytes
<code>TAMANO_CACHE_GLOBAL n</code>	<code>n ≥ 0</code>	Capacidad del caché global
<code>NEW_PROCESS id memoria</code>	<code>id</code> único, <code>memoria ≥ 1</code>	Crea un proceso y le asigna las páginas necesarias
<code>END_PROCESS id</code>	<code>id</code> existente	Libera todos los recursos del proceso
<code>ACCESS id direccion</code>	<code>direccion ≥ 0</code>	Referencia una dirección de memoria virtual del proceso
<code>ALLOCATE id memoria</code>	<code>memoria ≥ 1</code>	Asigna memoria adicional al proceso
<code>FREE id direccionInicial bytes</code>	<code>bytes ≥ 1</code>	Libera un rango contiguo
<code>STATUS</code>	sin argumentos	Imprime el estado completo del sistema
<code>METRICS</code>	sin argumentos	Imprime las métricas acumuladas
<code>HELP</code>	sin argumentos	Muestra la lista de comandos
<code>EXIT</code>	sin argumentos	Termina el simulador (modo interactivo)

Las líneas vacías y las que empiezan con `#` se ignoran (comentarios). Las tres directivas iniciales (`MEMORIA_TOTAL` , `TAMANO_PAGINA` , `TAMANO_CACHE_GLOBAL`) deben aparecer antes de la primera operación con procesos; el simulador detecta la combinación y configura el sistema automáticamente.

Ejemplo de archivo de entrada:

```
MEMORIA_TOTAL 65536
TAMANO_PAGINA 1024
TAMANO_CACHE_GLOBAL 8
NEW_PROCESS P1 4096
NEW_PROCESS P2 8192
ACCESS P1 100
ACCESS P1 2500
ACCESS P1 2500
ALLOCATE P1 2048
FREE P2 2048 2048
END_PROCESS P1
END_PROCESS P2
```

8. Formato de salida

La salida es estrictamente textual y se imprime por `std::cout` . Cada operación produce líneas con un formato consistente que facilita su lectura tanto por humanos como por scripts de verificación. Existen cuatro categorías de salida:

8.1 Mensajes de operación

```
NEW_PROCESS P1 OK: 4 paginas asignadas. Cache local max = 1
ACCESS P1 2500 -> VP=2 offset=452
    Resultado: HIT en cache LOCAL. Pagina fisica = 2
ALLOCATE P1 OK: 2 paginas adicionales. Total ahora: 6. Cache local max = 1
FREE P2 OK: 2 paginas liberadas. Cache local max = 1
END_PROCESS P1 OK: 6 paginas liberadas.
```

8.2 Mensajes de error

Todos los errores se muestran con el prefijo `[ERROR]` :

```
[ERROR] Proceso duplicado: P1
[ERROR] Memoria insuficiente para P2 (requiere 9 paginas, libres 4)
[ERROR] Pagina virtual 976 no asignada al proceso P1
```

8.3 Estado del sistema (comando `STATUS`)

Reporta memoria total, páginas libres y ocupadas, porcentaje de utilización y, para cada proceso, su memoria solicitada, sus páginas, su caché local y el contenido del caché global.

8.4 Métricas (comando `METRICS`)

```
===== METRICAS =====
Total accesos:           7
Hits en cache local:     0
Misses en cache local:   7
Hits en cache global:    0
Misses en cache global:  7
Hits en Splay (proceso): 7
Page faults:             0
Reemplazos LRU:          9
Errores detectados:      0
Paginas asignadas:       10/16
Paginas libres:          6
Utilizacion memoria:     62.5%
Hit-rate local:          0%
Hit-rate global:         0%
=====
```

9. Casos de prueba

El proyecto incluye tres archivos de prueba en la carpeta `input/`. Cada uno enfatiza una dimensión distinta del simulador y todos se ejecutan con `./simulador.exe input/<archivo>.txt`.

9.1 Caso 1 — Operaciones básicas (`test1.txt`)

Reproduce el ejemplo del enunciado (sección 1.1.8): configura un sistema de 64 páginas, crea dos procesos, ejecuta accesos repetidos, hace `ALLOCATE` y `FREE`, finaliza ambos procesos y reporta el estado.

Resultado esperado	Resultado obtenido
Sistema configurado con 64 páginas físicas, 2 procesos creados sin errores	✓
Segundo acceso a la misma dirección produce HIT local	✓
ALLOCATE aumenta las páginas y el tamaño del caché local	✓
END_PROCESS libera correctamente y METRICS muestra el conteo total	✓

9.2 Caso 2 — Manejo de errores (test2_errores.txt)

Provoca de forma deliberada los nueve tipos de error que el simulador debe detectar.

Error provocado	Mensaje esperado	Detectado
Proceso duplicado	[ERROR] Proceso duplicado: P1	✓
Proceso inexistente	[ERROR] Proceso inexistente: P9	✓
Dirección inválida	[ERROR] Direccion invalida: -50	✓
Memoria insuficiente	[ERROR] Memoria insuficiente ...	✓
Página no asignada	[ERROR] Pagina virtual ... no asignada	✓
FREE sobre página no asignada	[ERROR] FREE: pagina virtual ... no asignada	✓
Operación desconocida	[ERROR] Operacion desconocida: HACER_MAGIA	✓
Tamaño de página inválido	(intentar reconfigurar con procesos vivos)	✓
Argumento numérico inválido	[ERROR] Argumento numerico invalido	✓

9.3 Caso 3 — Política LRU (test3_lru.txt)

Configura un caché global pequeño (3 entradas) y un proceso de 10 páginas (caché local de 2). Realiza accesos secuenciales a 6 páginas distintas y luego vuelve a la primera, forzando reemplazos.

Resultado esperado	Resultado obtenido
Reemplazos LRU > 0 después de llenar los cachés	9 reemplazos detectados ✓
El último acceso a la página 0 produce MISS en ambos cachés	✓
Las páginas 4 y 5 quedan en el caché global (las más recientes)	✓
Las páginas más antiguas son las víctimas	✓

10. Manejo de errores

El simulador detecta y reporta los siguientes errores de forma uniforme con el prefijo `[ERROR]` . Todos incrementan el contador `errors` que se reporta en `METRICS` .

Error	Cuándo se dispara	Recuperación
Proceso inexistente	Al operar sobre un id no creado	El sistema continúa, la operación se descarta
Proceso duplicado	<code>NEW_PROCESS</code> con id existente	El nuevo proceso no se crea; el existente queda intacto
Memoria insuficiente	Páginas libres < páginas requeridas	No se asigna nada
Dirección inválida	<code>ACCESS</code> o <code>FREE</code> con dirección negativa o página inexistente	Acceso descartado
Liberación inválida	<code>FREE</code> sobre páginas no asignadas	Se ignora esa página y sigue con las siguientes
Tamaño de página inválido	<code>MEMORIA_TOTAL</code> no múltiplo de <code>TAMANO_PAGINA</code>	No se configura
Reconfiguración con procesos vivos	Cambiar parámetros con procesos activos	Se rechaza y se mantienen los valores anteriores
Operación desconocida	Comando no reconocido	Se ignora la línea
Argumento numérico inválido	Token donde se espera número	Se ignora la línea

Esta política de **manejo defensivo** garantiza que un comando inválido no pueda dejar el sistema en estado inconsistente: el AVL de páginas libres, las tablas de páginas y los cachés siempre quedan en un estado coherente después de procesar cualquier línea.

11. Análisis de complejidad

A continuación se describe la complejidad temporal de cada operación principal. Se utilizan los siguientes símbolos:

- **n**: número total de páginas físicas del sistema.
- **m**: número de páginas asignadas al proceso involucrado.
- **k**: número de páginas afectadas por la operación.
- **p**: número de procesos activos.

Operación	Mejor caso	Caso promedio / amortizado	Peor caso	Estructura dominante
NEW_PROCESS con k páginas	$\Omega(k)$	$O(k \cdot \log n)$	$O(k \cdot \log n)$	AVL (extraer mínimos) + Splay (insertar)
END_PROCESS con k páginas	$\Omega(k)$	$O(k \cdot \log n + g)$	$O(k \cdot \log n + g)$	Recorrido del Splay + devolución al AVL + recorrido del caché global (g = tamaño global)
ACCESS (HIT local)	$\Omega(1)$	$O(1)$	$O(1)$	Caché local (hash + lista)
ACCESS (HIT global)	$\Omega(1)$	$O(1)$	$O(1)$	Caché global (hash + lista)
ACCESS (MISS, va al Splay)	$\Omega(1)$	$O(\log m)$ amortizado	$O(m)$ un solo acceso	Splay (splay-to-root)
ALLOCATE con k páginas	$\Omega(k)$	$O(k \cdot \log n)$	$O(k \cdot \log n)$	Igual que NEW_PROCESS
FREE con k páginas	$\Omega(k)$	$O(k \cdot \log m)$	$O(k \cdot \log m)$	Por página: Splay + AVL
STATUS	$\Omega(n + p \cdot m)$	$\Theta(n + p \cdot m)$	$\Theta(n + p \cdot m)$	Recorridos in-order
METRICS	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Solo lectura de contadores
Cualquier operación del caché LRU	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	Lista doblemente enlazada + hash map

11.1 Análisis amortizado del Splay Tree

El **acceso a una página** dentro del Splay Tree de un proceso es $O(\log m)$ en sentido amortizado. Esto significa que si se realizan q operaciones cualesquiera (inserciones, búsquedas, eliminaciones), el costo total es $O(q \cdot \log m)$, aunque alguna operación aislada pueda costar hasta $O(m)$. La demostración clásica usa el método del potencial, que está fuera del alcance de este documento, pero la consecuencia práctica es muy importante: **los accesos repetidos a la misma página cuestan $O(1)$** después del primer acceso, porque la página queda en la raíz.

11.2 Complejidad espacial

Concepto	Memoria	Comentario
AVL de páginas libres	$O(n)$	Un nodo por página física libre
Splay Tree por proceso	$O(m)$	Un nodo por página asignada
Caché local	$O(0.20 \cdot m)$	Hasta el 20% de las páginas asignadas

Concepto	Memoria	Comentario
Caché global	$O(c)$	<code>c = TAMANO_CACHE_GLOBAL</code>
Tabla global de procesos	$O(p)$	Un <i>bucket</i> por proceso
Total	$O(n + p \cdot m + c)$	

12. Conclusiones

El simulador de memoria paginada implementado demuestra el funcionamiento integrado de dos estructuras de datos no triviales —el Árbol AVL y el Splay Tree— sobre un caso de uso real del cómputo: la administración de memoria en un sistema operativo. La separación de responsabilidades entre las siete clases (Page, AVLTree, SplayTree, Cache, Process, MemoryManager y OperationParser) permitió desarrollar y depurar cada componente de forma independiente, y conectar las partes mediante interfaces claras.

La elección del **AVL para las páginas libres** y del **Splay Tree para las tablas de páginas** resultó adecuada: el AVL ofrece garantías sólidas para un conjunto donde no hay un patrón de acceso predecible (las páginas se piden y se liberan en cualquier orden), mientras que el Splay aprovecha la localidad temporal de los accesos a memoria sin necesidad de mantener información extra de balanceo. Las pruebas ejecutadas confirman que ambos árboles funcionan correctamente, las métricas son coherentes con los accesos realizados y el sistema detecta nueve tipos distintos de error sin quedar en estados inconsistentes.

La implementación del **caché LRU en $O(1)$** mediante la combinación de lista doblemente enlazada y *hash map* fue una decisión técnica importante porque mantuvo el costo total de un `ACCESS` con HIT en tiempo constante. La política del caché local (`max(1, floor(paginasAsignadas * 0.20))`) se ajusta dinámicamente con `ALLOCATE` y `FREE`, manteniendo la proporción exigida por el enunciado.

Finalmente, durante el desarrollo surgió un problema técnico interesante: el conflicto entre el nombre de la clase `Process` y el header del sistema `<process.h>` de Windows, que se resolvió renombrando el archivo a `Proceso.h` y documentando la decisión en el código. Esta clase de inconvenientes, propios del trabajo con C++ en entornos *case-insensitive*, refuerzan la importancia de validar el código en el entorno objetivo desde etapas tempranas.

Como posibles extensiones futuras se podrían incorporar las funcionalidades sugeridas en el enunciado (sección 1.1.18): exportación de estadísticas a CSV, visualización gráfica del estado de memoria, pruebas unitarias automatizadas y otras políticas de reemplazo (Second-Chance, NRU, MRU) para comparar empíricamente sus desempeños.

Fin del documento técnico.